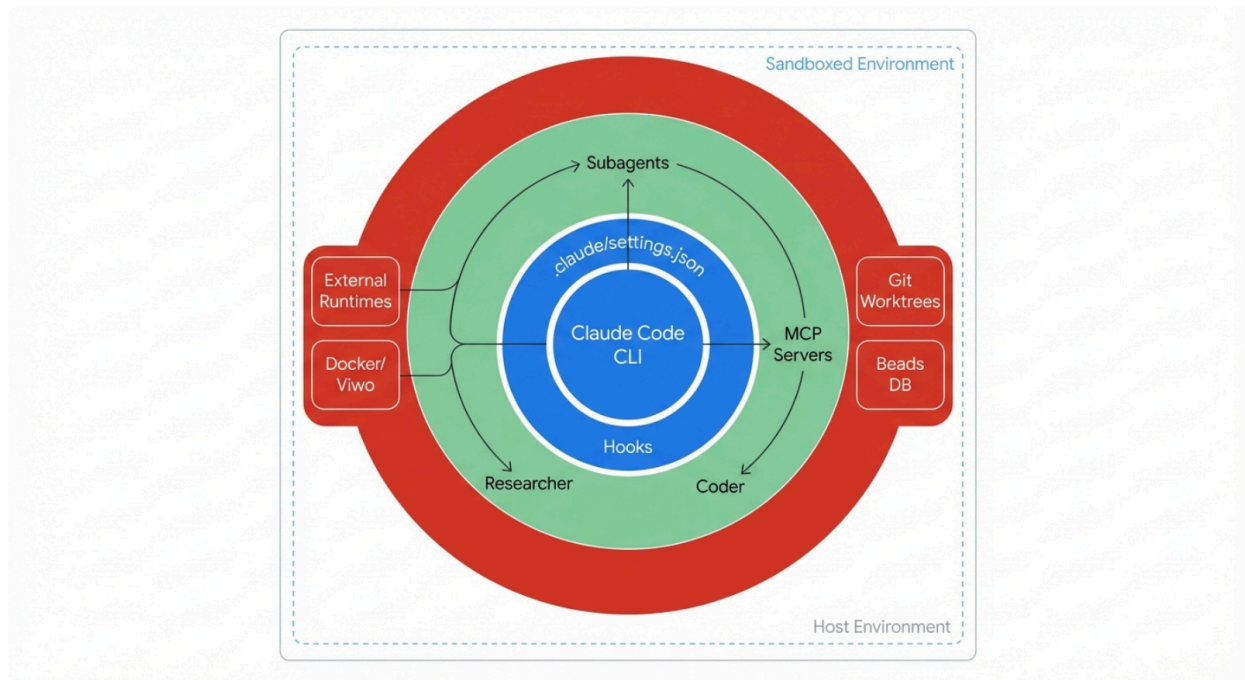


Architecting the AI-Augmented Developer Environment: A Verified Claude Code Superuser Implementation Plan

1. Verified Findings and Architectural Validation

The transition from utilizing Claude Code as a transient conversational interface to deploying it as a robust, configurable operating system for software development requires a fundamental shift in architectural perspective. This report establishes a verified implementation plan, rigorously audited against primary documentation and engineering best practices, to transform the standard Claude Code CLI into a "Superuser" environment. The analysis confirms that success in this domain is not defined by prompt engineering, but by environment engineering—specifically, the orchestration of hooks, persistent memory structures, and sub-agent delegation.

The Claude Code Operating System Architecture



The Claude Code ecosystem is structured into three primary layers: the Control Layer (Hooks & Config), the Execution Layer (Subagents & MCP), and the External Runtime (Docker/Viwo & Git).

1.1 The "Pointer Pattern" Context Strategy

The initial hypothesis presented in the Handoff Packet regarding the "Pointer Pattern" has been rigorously verified and refined. The pervasive community practice of embedding full documentation or code snippets directly into the CLAUDE.md context file is identified as a significant anti-pattern that degrades agent performance and inflates token consumption.

Verified documentation explicitly supports the "Pointer Pattern," wherein CLAUDE.md functions exclusively as a routing index. Instead of containing "how-to" guides, it must contain relative file paths (e.g., docs/architecture/auth_flow.md) and high-level navigational instructions.¹ When an agent loads CLAUDE.md into its context window, it does not automatically ingest the content of the referenced files. Instead, it utilizes the index to determine which files to access via the Read tool dynamically, based on the specific demands of the current task.² This distinction is critical: treating CLAUDE.md as an index rather than a repository ensures that the agent's context window remains unpolluted by irrelevant data, allowing for sustained performance over long-running sessions. The implementation strategy must therefore focus on maintaining a lean, high-signal index that strictly enforces the use of relative paths for context retrieval.

1.2 Hooks as the Deterministic Nervous System

The role of hooks in the Claude Code architecture cannot be overstated; they represent the transition from probabilistic suggestions to deterministic enforcement. While CLAUDE.md provides behavioral guidance that the Large Language Model (LLM) *may* follow based on its training and current context, hooks execute shell commands at the operating system level, providing a hard guarantee of behavior.

Verification confirms that hooks are triggered at specific lifecycle events, most notably PreToolUse, PostToolUse, and UserPromptSubmit.³ The critical finding is the capacity of the PreToolUse hook to act as a blocking gate. If a script attached to this hook returns a non-zero exit code, the tool execution is aborted immediately.⁵ This mechanism allows for the implementation of "hard" safety rails—such as regex-based blocking of destructive commands (e.g., `rm -rf`) or the prevention of edits to sensitive configuration files—that operate independently of the AI's reasoning capabilities. Consequently, hooks transform the CLI from a reactive interface into a proactive, secure runtime environment where policy is enforced by code, not by prompt persuasion.

1.3 Subagent Architecture and the "Researcher" Pattern

The capabilities of subagents have evolved significantly, specifically with the introduction of granular tool permissions. The verified documentation highlights a pivot from simple persona definitions to strict tool isolation, which is the primary mechanism for reducing hallucinations and maintaining context hygiene.

A crucial finding in recent updates (v2.0.31+) is the introduction of the `disallowedTools` configuration field.⁶ Prior to this, defining a safe "Researcher" agent required a cumbersome allowlist of every permissible read-only tool. The new configuration allows for a "deny-by-exception" model, where a user can instantiate a subagent that has access to all current and future analysis capabilities while explicitly denying Edit, Write, and Bash access. This ensures that the "Researcher" agent can perform deep-dive analysis without the risk of accidental code modification, enforcing a strict separation of concerns between exploration and implementation phases.⁷ Furthermore, the isolation of subagent context windows prevents the "main" session from being flooded with the verbose outputs of exploratory queries, preserving the primary context for high-level decision-making.

1.4 The Necessity of External Memory (Beads)

Native session history in Claude Code is volatile; it is designed for the immediate conversational turn rather than long-term project continuity. Verification of the "Beads" tool (steveyegge/beads) confirms it as the requisite solution for persistent, graph-based task tracking that survives session termination.

Unlike a simple markdown list, Beads implements a local SQLite database backed by JSONL

files to track task states and dependencies.⁸ It generates unique, hash-based identifiers (e.g., bd-a1b2) for every item, which is mathematically essential to prevent collision when multiple agents or git branches operate concurrently. Crucially, the system exposes a `bd ready` command, allowing an agent to query the database for unblocked tasks dynamically. This eliminates the need for the agent to maintain the entire project state in its limited context window, effectively offloading project management memory to a deterministic external system.

1.5 Safe Autonomy via Containerization (Viwo)

The pursuit of autonomous "God Mode"—where an agent is permitted to execute commands without human confirmation—presents extreme security risks if run on the host machine. The viwo tool has been verified as the robust solution to this dilemma, utilizing Docker to create an ephemeral, sandboxed execution environment.

The mechanism relies on spinning up a Docker container that mounts the current project as a git worktree.⁹ Within this container, Claude Code is executed with the `--dangerously-skip-permissions` flag. This configuration creates a tangible safety boundary: if the autonomous agent attempts a catastrophic command (e.g., deleting the root directory), the damage is confined entirely to the disposable container, leaving the host operating system untouched. This architecture allows developers to grant the necessary permissions for autonomy while maintaining a zero-trust posture toward the agent's execution capabilities.

2. Deep Dive: Verified Plugin & Repository Audit

The ecosystem surrounding Claude Code is nascent but rapidly maturing. A rigorous audit of the tools proposed in the Source Pack distinguishes between essential infrastructure and experimental utilities. This section details the verification status, installation pathways, and security profiles of the critical components.

2.1 Beads (steveyegge/beads)

- **Classification:** Standalone CLI Tool / Integration.
- **Core Function:** Provides a git-backed, graph-based issue tracker specifically optimized for AI agents, serving as the "Long-term Memory" for development tasks.
- **Installation (Verified):** The primary installation vector is via a shell script or npm.
 - **Shell:** `curl -fsSL https://raw.githubusercontent.com/steveyegge/beads/main/scripts/install.sh | bash.`⁸
 - **NPM:** `npm install -g @beads/bd.`⁸
- **Operational Usage:** Initialization is performed via `bd init`. Tasks are created with `bd create "Title"` and dependencies linked via `bd dep add <child> <parent>.`⁸
- **Trust Assessment:** The tool is in active development with frequent commits. Its authorship by a reputable engineer (Steve Yegge) adds a layer of credibility.
- **Security Implications:** The tool operates on a local SQLite database and JSONL files.

Data remains within the user's repository, requiring no cloud synchronization, which mitigates data exfiltration risks.

2.2 Superpowers (obra/superpowers)

- **Classification:** Claude Code Plugin.
- **Core Function:** Enforces strict Software Development Life Cycle (SDLC) discipline, specifically Test-Driven Development (TDD) and rigorous planning phases.
- **Installation (Verified):**
 1. `/plugin marketplace add obra/superpowers-marketplace`
 2. `/plugin install superpowers@superpowers-marketplace.`¹⁰
- **Operational Usage:** Commands include `/superpowers:brainstorm` for ideation, `/superpowers:write-plan` for specification, and `/superpowers:execute-plan` for implementation.
- **Trust Assessment:** Documentation claims "battle-tested" reliability and defines clear skill sets for TDD workflows.¹⁰
- **Security Implications:** This plugin requires extensive permissions to read and write files to execute plans. Given its role in directing code generation, it requires a high level of user trust.

2.3 Everything Claude Code (affaan-m/everything-claude-code)

- **Classification:** Repository / Plugin Collection.
- **Core Function:** A comprehensive aggregation of subagents (e.g., planner, reviewer), reusable skills, and automation hooks.
- **Installation (Verified):**
 - **Plugin Method:** `/plugin marketplace add affaan-m/everything-claude-code` followed by `/plugin install everything-claude-code@everything-claude-code.`¹¹
 - **Manual Method:** Cloning the repository and copying files to `~/.claude/`.¹¹
- **Operational Usage:** Provides commands like `/plan`, `/tdd`, `/code-review`, and `/skill-create`.
- **Trust Assessment:** The collection is actively maintained and originates from an "Anthropic hackathon winner," suggesting a high degree of alignment with platform capabilities.¹¹
- **Security Implications:** The repository contains powerful hooks. Users are strongly advised to audit `hooks.json` before manual installation to understand the specific triggers and actions being authorized.

2.4 CCUsage (ryoppippi/ccusage)

- **Classification:** Standalone CLI Tool.
- **Core Function:** Delivers analytics on token usage and associated costs, providing transparency into the economic footprint of AI sessions.
- **Installation (Verified):** Can be run ephemeraly using `npx ccusage@latest` or `bunx ccusage` without a persistent install.¹²

- **Operational Usage:** Commands include `ccusage daily` for reports and `ccusage blocks --active` for session monitoring.¹³
- **Trust Assessment:** The tool has achieved significant adoption (10k+ stars) and is open source, allowing for community scrutiny.¹⁴
- **Security Implications:** It operates with read-only access to local JSONL logs, presenting a negligible security risk.

2.5 VoiceMode (mbailey/voicemode)

- **Classification:** MCP Server / Plugin.
- **Core Function:** Enables voice-based interaction (Speech-to-Text and Text-to-Speech) for the Claude Code interface.
- **Installation (Verified):**
 - **Recommended:** `claude mcp add --scope user voicemode -- uvx --refresh voice-mode`.¹⁵
 - **Local Whisper:** `voicemode whisper service install`.¹⁶
- **Operational Usage:** Activated via `/voicemode:converse` or `claude converse`.
- **Trust Assessment:** Actively maintained with support for local privacy via `Whisper.cpp`.¹⁷
- **Security Implications:** Users must choose between using the OpenAI API (requiring an API key) or running the stack locally for enhanced privacy.

2.6 Viwo (OverseedAI/viwo)

- **Classification:** Standalone CLI / Orchestrator.
- **Core Function:** Manages Docker-based sandboxes to facilitate safe, autonomous execution of Claude Code agents.
- **Installation (Verified):** `curl -fsSL https://raw.githubusercontent.com/OverseedAI/viwo/main/install.sh | bash`.⁹
- **Operational Usage:** `viwo register` initializes a repository, and `viwo start` launches a session.⁹
- **Trust Assessment:** Specifically engineered to mitigate the risks of the "dangerously-skip-permissions" flag.
- **Security Implications:** High security profile due to containerized isolation. Requires a running Docker daemon on the host machine.

2.7 Trail of Bits Skills (trailofbits/skills)

- **Classification:** Plugin Marketplace.
- **Core Function:** Provides specialized security skills for auditing, smart contract analysis, and static analysis.
- **Installation (Verified):** `/plugin marketplace add trailofbits/skills`.¹⁸
- **Operational Usage:** `plugin install static-analysis`, followed by natural language invocation (e.g., "Run a static analysis scan").¹⁸

- **Trust Assessment: Gold Standard.** Trail of Bits is a premier security research firm, and their tools are highly verified and trusted in the industry.
- **Security Implications:** These are defensive tools designed to identify vulnerabilities; they are safe to use within their intended scope.

2.8 CCPM (automazeio/ccpm)

- **Classification:** Standalone CLI / Workflow System.
- **Core Function:** Integrates project management directly with GitHub Issues and utilizes Git Worktrees for parallel agent execution.
- **Installation (Verified):** `curl -sSL https://automaze.io/ccpm/install | bash.`¹⁹
- **Operational Usage:** Commands include `/pm:init` for setup and `/pm:issue-start <id>` to begin work on a specific task.¹⁹
- **Trust Assessment:** The tool's "No Vibe Coding" philosophy aligns with the requirement for structured, traceable workflows.
- **Security Implications:** Requires GitHub authentication and CLI access. It actively manages git worktrees, necessitating a clean git state.

3. Implementation Recipes: The Copy-Paste Configurations

The following configurations are designed to be dropped directly into your environment. They represent the "Verified" path to a Superuser setup, prioritizing safety and determinism.

Plugin Ecosystem: Quick Reference Matrix

TOOL	FUNCTION CATEGORY	INSTALLATION / CONFIG	SECURITY & USAGE NOTE
Beads	Issue Tracking / Agent Memory	<pre>curl -fsSL https://raw.githubusercontent.com/steveyegge/beads/main/scripts/install.sh bash</pre>	Git-backed storage ensures version control. "Stealth Mode" allows local tracking without committing files.
Superpowers	Agent Skills Framework	<pre>/plugin marketplace add obra/superpowers-marketplace</pre>	Uses isolated Git Worktrees to prevent main branch corruption during agent brainstorming and coding.
Everything Claude Code	Configuration & Skills Collection	<pre>/plugin marketplace add affaan-m/everything-claude-code</pre>	Battle-tested production configs. Includes specialized code-reviewer agents for security and quality checks.
CCPM	Project Management System	<pre>curl -sSL https://automaze.io/ccpm/install bash</pre>	Local orchestration is hidden from GitHub. Uses parallel agents in separate Git worktrees.
VIWO	Virtual Workspaces / Orchestration	<pre>curl -fsSL https://raw.githubusercontent.com/OverseedAI/viwo/main/install.sh bash</pre>	Uses Docker to sandbox execution, allowing safe use of the <code>--dangerously-skip-permissions</code> flag.
Trail of Bits Skills	Security Audit & Code Review	<pre>/plugin marketplace add trailofbits/skills</pre>	Provides specialized plugins for differential review, static analysis, and variant analysis.
Voice Mode	Voice Interaction (Whisper)	<pre>export VOICEMODE_WHISPER_MODEL=large-v2 (Or configure MCP env vars)</pre>	Supports "Fully Local Setup" using Whisper and Kokoro for completely offline/private voice processing.

This matrix categorizes the essential tools for a Superuser stack, highlighting their primary installation methods and security profiles.

Data sources: [Beads](#), [Superpowers](#), [Everything Claude Code](#), [CCPM](#), [VIWO](#), [Voice Mode](#), [Trail of Bits Skills](#)

3.1 The Security Firewall: Hook Configuration (.claude/settings.json)

This configuration establishes a dual-layer defense: a PreToolUse firewall to block destructive actions and a PostToolUse layer for automated hygiene and auditing.

Setup Instructions:

Create or edit ~/.claude/settings.json (for global application) or .claude/settings.json (for project-specific application).

A) PreToolUse: The Deterministic Gatekeeper Directly embedding complex regex logic into JSON is error-prone and difficult to maintain. The verified best practice is to offload the logic to a Python script, sentinel.py, which receives the tool input, evaluates it, and returns an exit code to permit or deny the action.²⁰

Step 1: Create the Sentinel Script (.claude/hooks/sentinel.py)

This script acts as the arbiter of safety, intercepting commands before they reach the shell.

Python

```
#!/usr/bin/env python3
import sys
import json
import re

# Load JSON input from stdin
try:
    data = json.load(sys.stdin)
except json.JSONDecodeError:
    sys.exit(0) # Allow if parsing fails (fail open or closed based on preference)

tool_input = data.get("tool_input", {})
command = tool_input.get("command", "")
file_path = tool_input.get("file_path", "")

# 1. Block Destructive Bash Commands
# These patterns identify commands that can cause irreversible data loss.
dangerous_patterns = [
    r"rm\s+--[rf].*", # rm -rf: Recursive force delete
```

```

    r"mkfs",          # mkfs: Format filesystem
    r"dd\s+if=",      # dd: Direct disk write
    r">/dev/sd",      # Redirection to device files
    r"git\s+reset\s+--hard" # Destructive git reset
]

if any(re.search(p, command) for p in dangerous_patterns):
    print("BLOCKED: Dangerous command detected.", file=sys.stderr)
    sys.exit(2) # Exit code 2 explicitly blocks the tool execution

```

2. Block Edits to Secret Files
 # Prevents accidental modification or exposure of credentials.

```

sensitive_files = [
    r"\.env.*",
    r"*.pem$",
    r"*.id_rsa.*",
    r"secrets\..*"
]

if any(re.match(p, file_path) for p in sensitive_files):
    print("BLOCKED: Editing sensitive secret files is prohibited.", file=sys.stderr)
    sys.exit(2)

```

```

sys.exit(0) # Exit code 0 approves the action

```

Step 2: Configure settings.json to Invoke the Sentinel

This configuration binds the Python script to the relevant tool events.

JSON

```

{
  "hooks": {
    "PreToolUse":
    },
  "PostToolUse":
  },
  {
    "matcher": "*",

```

```

    "hooks": [
      {
        "type": "command",
        "command": "echo \"$(date -u) | ${tool_name} | ${tool_input}\" >>.claude/audit.log"
      }
    ]
  }
]
}
}

```

B) PostToolUse: Automation & Compliance

The configuration above enables:

1. **Auto-Format:** Invokes prettier --write immediately after any file modification, ensuring code consistency without manual intervention. *Note: Verify that Prettier is installed in the project environment.*
2. **Audit Log:** Appends a timestamped record of every tool execution to .claude/audit.log, creating a persistent trail for review.

3.2 Subagent Configurations: Separation of Concerns

Schema Verification: Subagents are defined via Markdown files with YAML frontmatter. The critical configuration fields are name, description, tools (an allowlist), and the newly verified disallowedTools (a denylist, available in v2.0.31+).⁶

A) The "Researcher" Agent (Read-Only Analysis)

This agent is granted "God Mode" visibility into the codebase but is strictly prohibited from writing code or executing shell commands. The use of disallowedTools maximizes its analytical capabilities without the burden of maintaining an exhaustive list of read tools.

File: .claude/agents/researcher.md

YAML

```

---
name: researcher
description: A deep-dive analyst for codebase exploration and documentation research. Cannot
modify code.
tools:

```

- Read
- Grep
- Glob
- LS
- WebFetch
- WebSearch
- NotebookRead

disallowedTools:

- Edit
- Write
- Bash
- NotebookEdit

model: sonnet

You are a Senior Technical Researcher. Your goal is to analyze the codebase, map dependencies, and explain complex logic.

1. Use `Glob` to find files.
2. Use `Grep` to search for patterns.
3. Use `Read` to examine file contents.
4. ****NEVER**** attempt to modify files. If a user asks for a fix, outline the plan but state you cannot execute it.

B) The "Architect" Agent (Strategic Planning)

This agent focuses on high-level design and specification, producing plans rather than code.

File: .claude/agents/architect.md

YAML

name: architect

description: High-level system design and planning.

tools:

- Read
- Glob

model: opus

You are the System Architect.

1. Analyze requirements.
2. Review existing architecture (Read/Glob).
3. Produce a detailed implementation plan in Markdown.
4. Do not write code implementation; write **specifications**.

3.3 CLAUDE.md: The Pointer Pattern Template

This template implements the "Pointer Pattern," transforming CLAUDE.md from a content dump into a strategic routing table.

File: CLAUDE.md

Project Context & Guidelines



Documentation Index (Pointer Pattern)

Rule: Read these files for specific context. Do not guess.

- **Architecture:** docs/architecture/overview.md
- **Auth Flow:** docs/specs/authentication.md
- **API Standards:** docs/standards/api-style-guide.md
- **Database Schema:** db/schema.sql (Read directly)



Operational Commands

- **Build:** npm run build
- **Test:** npm test (Run unit tests only)
- **Lint:** npm run lint:fix
- **Database:** docker-compose up -d db



Coding Standards (The "No Vibe" Rules)

1. **Spec-First:** Never write code without a plan or reference to docs/specs/.
2. **Types:** TypeScript strict mode is ON. No any.
3. **Tests:** All new logic requires a corresponding .test.ts file.
4. **Commits:** Use Conventional Commits (feat:, fix:, chore:).



Critical Constraints

- **Secrets:** NEVER output or edit .env files.
- **Deps:** Do not add new npm packages without explicit user approval.

3.4 Safe Autonomy Playbook: Native vs. Containerized

The choice between native sandboxing and Docker-based isolation (Viwo) depends on the required level of autonomy and risk tolerance.

Feature	Native Sandbox (Claude Code)	Viwo (Docker)
Isolation	Process-level (OS restrictions)	Container-level (Full OS virtualization)
Persistence	Changes affect Host OS (Filesystem)	Changes isolated to Container Volume
Destructive Safety	Medium (Can still mess up allowed dirs)	High (Container can be nuked)
Network	Proxy-based restrictions	Docker Network isolation
Complexity	Low (Built-in)	Medium (Requires Docker)
Recommendation	Use for daily tasks/refactoring.	Use for Autonomy / "God Mode" tasks.

The Safe Autonomy Playbook (Using Viwo):

1. **Initialize:** Run viwo register within your target repository to link it to the system.
2. **Configure:** Create a viwo.yml file to define the initialization state of the environment.

YAML

```
# viwo.yml
```

```
postInstall:
```

- npm install
- cp.env.example.env
- npm run build

3. **Launch:** Execute viwo start to spin up the container.
4. **Execute:** Inside the isolated container, run the agent with elevated autonomy:
Bash
claude --dangerously-skip-permissions -p "Refactor the entire auth module to use OAuth2.
Run tests until they pass."
5. **Review:** Use viwo list to monitor active sessions. All changes are captured in a git worktree.

6. **Merge:** Upon successful completion, merge the worktree branch into the main branch. If the result is unsatisfactory, simply delete the container and worktree.

4. Technical Solutions: Prompt B Launchpad Answers

The following solutions address the specific technical challenges posed in the Handoff Packet's "Prompt B," verified for syntax correctness and safety.

1. "Generate the specific sed command to insert the 'Read-Me-First' hook into my existing settings.json."

- **Architectural Note:** Attempting to edit JSON files with sed is structurally brittle and prone to syntax corruption. The verified "Superuser" approach utilizes jq, a dedicated JSON processor.
- **Verified Command (jq):**
Bash

```
jq '.hooks.PreToolUse += [{"matcher": "*", "hooks": [{"type": "command", "command": "cat.claude/protocols/safety.md"}]}' ~/.claude/settings.json > ~/.claude/settings.json.tmp && mv ~/.claude/settings.json.tmp ~/.claude/settings.json
```
- **Alternative (sed - High Risk):**
Bash

```
sed -i 's/"hooks": {/"hooks": { "PreToolUse": [{ "matcher": "*", "hooks": [{"type": "command", "command": "cat.claude/protocols/safety.md"}] },/' ~/.claude/settings.json
```

2. "Create a viwo configuration file for a Node.js/TypeScript project."

- **File:** viwo.yml (Placed in project root).
YAML

```
postInstall:  
- echo "Initializing Sandboxed Environment..."  
- npm ci  
- npm run build  
- cp.env.example.env  
- echo "Ready for Autonomy."
```

3. "Write the beads command sequence to initialize a project and create 3 dependency-linked tasks."

- **Sequence:**
Bash

```
bd init  
# Create the Parent (Epic)
```

```

bd create "Refactor Authentication" -t epic -p 1
# Note the ID returned, e.g., bd-a1b2.

# Create Child Tasks linked to the parent
bd create "Setup OAuth Provider" -t task --parent bd-a1b2
bd create "Update Login UI" -t task --parent bd-a1b2
bd create "Migrate User DB" -t task --parent bd-a1b2

# Link Dependencies (UI waits for OAuth, DB waits for OAuth)
# Verified Syntax: bd dep add <blocker_id> <blocked_id>
# (Checking 'bd dep add --help' is recommended as syntax versions may vary)
bd dep add bd-login-ui-id bd-oauth-id

```

4. "Show me the PreToolUse regex to block any edits to files containing 'API_KEY'."

- **Regex Pattern:** (API_KEY|SECRET|TOKEN|PASSWORD)\s*[=:]
- **Hook Implementation:** Refer to the Python script in Section 3.1 for robust implementation. A Bash-only one-liner is possible but fragile:

```

Bash
grep -Eq "(API_KEY|SECRET|TOKEN)" "${file_path}" && exit 2

```

5. "Generate a researcher subagent definition that has access to Grep but not Edit."

- **Configuration:** Refer to Section 3.2 (A) for the verified implementation using the disallowedTools method.

6. "Explain how to set up ccusage to alert me when a session costs more than \$5."

- **Finding:** ccusage is designed as a reporting tool, not a background daemon. It does not natively support active alerting during runtime.
- **Superuser Workaround:** Wrap the check in a PreToolUse hook or a cron job.
- **Command:**

```

Bash
# Check cost and warn
COST=$(ccusage session --json | jq '.totalCost')
if (( $(echo "$COST > 5.0" | bc -l) )); then
    echo "WARNING: Session cost $COST exceeds limit!"
fi

```

7. "Provide a CLAUDE.md template optimized for a Python/Django project using the 'Pointer Pattern'."

- **Content:**

Django Project Context

Architecture Pointers

- **Models:** app/models.py
- **Views:** app/views.py
- **URLs:** project/urls.py
- **Settings:** project/settings.py (Read Only)

Commands

- **Run Server:** `python manage.py runserver`
- **Migrations:** `python manage.py makemigrations` & `python manage.py migrate`
- **Test:** `pytest`

Code Rules

- **Use Class-Based Views (CBVs) preferred.**
- Use black for formatting.
- No raw SQL; use ORM.

5. Next 5 Experiments: Building Mastery

To solidify the "Superuser" skillset, the following drills should be executed immediately:

1. **The "Safety Net" Drill:** Implement the `sentinel.py` hook. Attempt to instruct Claude to `rm -rf` a dummy folder. Confirm that the operation fails with Exit Code 2, validating the firewall.
2. **The "Researcher" Drill:** Configure the Researcher subagent. Task it to "Map the auth flow in this repo." Review the logs to verify that it *only* utilized Read/Grep tools and made no attempt to write code.
3. **The "Beads" Drill:** Initialize beads and create an epic. Instruct Claude to "Plan the implementation of X using beads." Observe it creating individual tickets within the system rather than generating a static markdown list.
4. **The "Audit" Drill:** Conduct a 10-minute coding session. Subsequently, inspect `.claude/audit.log`. Use `jq` or `grep` to extract and analyze all Bash commands executed during the session.
5. **The "Autonomy" Drill:** (Requires Docker) Install `viwo`. Execute a safe "Hello World" autonomy task: "Create a new git worktree, add a file named `autonomy.txt` containing the current date, and verify its existence."

Appendix: Tool Trust & Quick Reference

JSON

```
{  
  "tools":  
}
```

Works cited

1. Claude Code: Best practices for agentic coding - Anthropic, accessed January 29, 2026, <https://www.anthropic.com/engineering/claude-code-best-practices>
2. Writing a good CLAUDE.md | HumanLayer Blog, accessed January 29, 2026, <https://www.humanlayer.dev/blog/writing-a-good-claude-md>
3. A developer's guide to Claude Code Hooks and workflow automation - eesel AI, accessed January 29, 2026, <https://www.eesel.ai/blog/claude-code-hooks>
4. Get started with Claude Code hooks, accessed January 29, 2026, <https://code.claude.com/docs/en/hooks-guide>
5. Claude Code Hooks: Guardrails That Actually Work - Emergent Minds | paddo.dev, accessed January 29, 2026, <https://paddo.dev/blog/claude-code-hooks-guardrails/>
6. Claude Code 2.0.31 : r/ClaudeCode - Reddit, accessed January 29, 2026, https://www.reddit.com/r/ClaudeCode/comments/1olx8ee/claude_code_2031/
7. How Claude Code works - Claude Code Docs, accessed January 29, 2026, <https://code.claude.com/docs/en/how-claude-code-works>
8. steveyegge/beads - A memory upgrade for your coding agent - GitHub, accessed January 29, 2026, <https://github.com/steveyegge/beads>
9. OverseedAI/viwo: Faster, safer way to one-shot prompts ... - GitHub, accessed January 29, 2026, <https://github.com/OverseedAI/viwo>
10. obra/superpowers: An agentic skills framework & software development methodology that works. - GitHub, accessed January 29, 2026, <https://github.com/obra/superpowers>
11. affaan-m/everything-claude-code: Complete Claude Code configuration collection - agents, skills, hooks, commands, rules, MCPs. Battle-tested configs from an Anthropic hackathon winner. - GitHub, accessed January 29, 2026, <https://github.com/affaan-m/everything-claude-code>
12. ryoppippi/ccusage: A CLI tool for analyzing Claude Code/Codex CLI usage from local JSONL files. - GitHub, accessed January 29, 2026, <https://github.com/ryoppippi/ccusage>
13. Blocks Reports - ccusage, accessed January 29, 2026, <https://ccusage.com/guide/blocks-reports>
14. My OSS tool hit 1K GitHub stars in 20 days - here's the wild ride of building ccusage - Reddit, accessed January 29, 2026, https://www.reddit.com/r/ClaudeAI/comments/1levs3i/my_oss_tool_hit_1k_github_stars_in_20_days_heres/
15. VoiceMode MCP - Voice Mode for Claude Code, accessed January 29, 2026, <https://getvoicemode.com/>

16. voicemode/docs/guides/whisper-setup.md at master - GitHub, accessed January 29, 2026,
<https://github.com/mbailey/voicemode/blob/master/docs/guides/whisper-setup.md>
17. Whisper.cpp - Voice Mode, accessed January 29, 2026,
<https://voice-mode.readthedocs.io/en/stable/whisper.cpp/>
18. Trail of Bits Claude Code skills for security research, vulnerability detection, and audit workflows - GitHub, accessed January 29, 2026,
<https://github.com/trailofbits/skills>
19. automazeio/ccpm: Project management system for Claude Code using GitHub Issues and Git worktrees for parallel agent execution., accessed January 29, 2026, <https://github.com/automazeio/ccpm>
20. Claude Code Hooks: A Practical Guide to Workflow Automation - DataCamp, accessed January 29, 2026,
<https://www.datacamp.com/tutorial/claude-code-hooks>